

MPLADS Risk Intelligence Platform

Complete project guide for a first-time reader

Read this first. It assumes you know nothing about the project. It tells you, in order: how to run it, what to click once it runs, and then how every part of it is built.

Everything in the project is **synthetic demo data**. No real people, no real government records.

Version 1.0 - prototype

How to read this guide

The guide has four blocks. You do not have to read it all in one go. Start at Part 2, get the project running, do the tour in Part 3, and come back to the later parts whenever you want to know how something works.

Part	What it answers	Read it when
1. What is this project?	The problem, the idea, who uses it	You want the big picture first
2. Running the project	Install, set up, start, log in, fix common problems	Right now - do this first
3. Your first 30 minutes	A guided click-through as each type of user	It is running and you want to see what it does
4. How it is built	The architecture in plain words, and what happens when you click a button	You want to understand the design
5. Folder-by-folder map	Every folder and file, and what it is for	You want to find something in the code
6. The database	All 18 tables, how to look inside the database	You want to see or query the data
7. Backend modules	Every Python module, every API endpoint, roles and permissions	You want to change or extend the server
8. Reading PDFs	How the app extracts and validates data from a PDF	You want to know how the PDF step works
9. The risk score	The exact formula, weights, comparison and machine learning	You want to know how a score is produced
10. The frontend	Every screen (33 of them), what it does, which data it uses	You want to know what each page is for
11. The seed data	How the demo data is created, how to reset it	You want to reset, change or understand the demo data
12. Everyday tasks	How-to recipes: add a user, change a weight, back up, run tests	You need to do something specific
13. Troubleshooting and FAQ	Problems and their fixes	Something is not working
14. Limits and ideas	What is a prototype, what is missing, what to build next	You plan to continue the project
Glossary	Every technical word used in this guide	A word is unfamiliar

Tip: Words written like **this** are things you type or file names. Words in **bold** are buttons or menu names you click.

Part 1 - What is this project?

1.1 The problem, in plain words

In India, the **MPLADS** scheme (Member of Parliament Local Area Development Scheme) gives every Member of Parliament money each year to pay for small development works in their area: a road, a community hall, a water pipeline, a school room, street lights. Government bodies such as municipal corporations, the Public Works Department and zilla parishads build these works. They are called **implementing agencies**.

Government officials have to keep an eye on hundreds of such projects. Some fall far behind schedule. Some spend money faster than the actual work progresses. Some keep asking for more time and more money. Officials cannot read every report, so they need help deciding **which projects deserve attention first**.

1.2 What this software does

This project is a working prototype of a monitoring tool. It does not accuse anyone. It looks at a project report, compares the project with earlier similar projects, and produces a **risk score from 0 to 100 with reasons** - so that a human official knows where to look first. The official then decides what to do, and every step is written to a permanent record.

The whole story, from a PDF to a decision, is one connected workflow:

1. **Log in.** The system looks up who you are in the database and shows the screens for your role.
2. **Upload a PDF.** An officer uploads a project report as a PDF file.
3. **Extraction.** The server reads the PDF and pulls out about 26 fields (project ID, budget, dates, progress ...) together with a confidence level and the page each value came from.
4. **Human verification.** The officer checks the values, corrects mistakes and confirms. Nothing is analysed until a human has verified it.
5. **Storage.** The verified project is saved in the PostgreSQL database.
6. **Historical comparison.** The project is compared with about 190 earlier projects: same category, district, agency, similar budget and duration.
7. **Risk analysis.** Ten rule checks plus a machine-learning "unusual pattern" detector (Isolation Forest) are combined into one score.
8. **Explanation.** Every point of the score is explained ("180-day delay", "progress below plan" ...).
9. **Notification.** The right people get a coloured notification.
10. **Officer decision.** The officer records a decision, the reason, the evidence reviewed and a follow-up.
11. **Escalation.** If needed the case goes to the Head Officer, who can acknowledge, ask for evidence, reopen, assign an inspection or close the review.
12. **Audit trail.** Everything above is stored in a tamper-evident log: who, what, when, old value, new value, reason.

Around this main workflow there are two more groups of people. **Citizens** can look at public projects on a map, review them and file complaints. **Agencies** report progress, ask for extra time or money, and answer questions from officers.

1.3 Four rules the project always follows

- **Synthetic data only.** Every person, project, complaint and PDF in the project was generated. Nothing is real.

- **A risk score is a signal, not an accusation.** The app never says a project is fraudulent. It says things like "unusual pattern detected" and "priority review recommended". The score bands (0-29 Low, 30-59 Medium, 60-79 High, 80-100 Critical) are **prototype thresholds, not official ones**, and the app says so on screen.
- **The role always comes from the database.** There is no "I am an officer" drop-down at login. Whoever you are in the database decides what you can see and do - on the screen and, more importantly, on the server.
- **Everything important is recorded.** Logins, uploads, corrections, analyses, decisions, notifications, status changes are all written to the audit log, and the database itself refuses to edit or delete audit records.

1.4 Who uses it

Role	Who this is	What they do in the app
Citizen	A member of the public	Search public projects and the map, see progress and public finances, write reviews, submit complaints (optionally anonymous), track them, give feedback, appeal. Citizens can sign up themselves.
Implementing Agency	Staff of the agency that builds the works	See only their own agency's projects, submit progress reports and expenditure, upload photos, request time extensions or budget changes, answer complaints and officer questions.
Officer	A monitoring officer	Upload and verify project PDFs, run the analysis, read the explanation, screen and resolve complaints, request clarifications and inspections, approve agency requests, record decisions, escalate, read the audit trail. Some officers are marked as field inspectors and carry out inspections.
Head Officer	The supervising senior	See escalated and high-risk cases, acknowledge, request evidence, reopen, assign inspections, add notes, escalate further, close reviews, decide citizen appeals, run the scheduled supervisory checks, check the audit summary.
Admin	The system administrator	Create users, change roles, disable accounts, reset passwords, view the audit overview. The admin cannot see or decide project cases.

Part 2 - Running the project

This part is written for **Windows**. Mac and Linux notes are at the end. Follow the steps in order. The whole thing takes about 20 minutes the first time, most of it waiting for downloads.

2.1 What you need before you start

You need	Why	Where to get it
Python 3.10 or newer	The server and the setup script are written in Python	python.org/downloads
PostgreSQL 12 or newer	The database that stores everything	postgresql.org/download/windows
A web browser	Chrome, Edge or Firefox	Already on your computer
Internet (first time only)	Setup downloads Python packages. The map also needs internet for its map tiles	-
About 1 GB free disk space	Python packages and the database	-

Note: You do **not** need Node.js or any programming knowledge to run the project. The screens are already built and included.

Install Python

1. Go to python.org/downloads and download the latest Python 3 installer for Windows.
2. Run the installer. On the very first screen, **tick "Add python.exe to PATH"** at the bottom, then click **Install Now**.
3. When it finishes, press the Windows key, type `cmd`, press Enter, and type `python --version`. You should see something like `Python 3.12.4`. If Windows says "Python was not found", close the black window and reinstall, making sure the PATH box was ticked.

Install PostgreSQL

1. Go to postgresql.org/download/windows, click **Download the installer**, and download the newest version for Windows x86-64.
2. Run it and click Next through the screens, keeping the defaults (the list of components, the install folder and the data folder).
3. When it asks for a **password for the database superuser (postgres)**, type a password you will remember and **write it down**. You need it in the next step.
4. Keep the port at **5432**. Keep the default locale. Finish the installer. You can skip "Stack Builder" at the end.
5. Check it is running: press Win+R, type `services.msc`, press Enter, and find a service whose name starts with **postgresql-x64**. Its status must be **Running**. If not, right-click it and choose Start.

Careful: If you forget the postgres password you cannot use that database. Write it down somewhere safe (it is only stored on your own computer).

2.2 Unzip the project

Right-click the zip file and choose **Extract All**. Put the folder somewhere simple such as `C:\mplads`. Avoid folders that are synced to the cloud (OneDrive) because the setup creates many small files. Open the extracted folder. You should see files like `setup.bat`, `run.bat`, `README.md` and folders like `backend` and `frontend`.

2.3 First-time setup (double-click setup.bat)

Double-click `setup.bat`. A black window opens and does everything in four steps. It asks you a few questions; press Enter to accept the value shown in square brackets.

Step you see	What it is doing
[1/4] Python environment	Creates a private folder <code>.venv</code> and downloads the Python packages the server needs (FastAPI, SQLAlchemy, scikit-learn, pandas, PDF readers ...). Takes a few minutes.
[2/4] PostgreSQL database	Asks for your PostgreSQL details, connects, creates a database called <code>mplads</code> , and writes a file called <code>.env</code> with the connection settings and a random secret key.
[3/4] Frontend	Checks that the ready-made screens (folder <code>frontend/dist</code>) exist. They are included, so it just says "nothing to do".
[4/4] Tables and demo data	Creates all 18 tables and fills them with the synthetic demo data (the "seed"). Takes 30-60 seconds.

The questions in step 2 are:

Question	What to type
Host	Press Enter (it means "this computer": localhost)
Port	Press Enter (5432)
User	Press Enter (postgres)
Password	Type the PostgreSQL password you chose. Nothing appears while you type - that is normal. Press Enter.
Database name to create/use	Press Enter (mplads)

When it prints **Setup complete**, you are done with setup. You only ever do this once.

Tip: If you make a typo in the password, just run setup again and add `--force` (open a command window in the folder and type `python setup.py --force`) so it asks the questions again.

2.4 Start the app (double-click run.bat)

1. Double-click `run.bat`. A black window opens and shows "MPLADS Risk Intelligence Platform -> `http://localhost:8000`".
2. After a few seconds your browser opens the login page. If it does not, open a browser yourself and go to `http://localhost:8000`.
3. **Leave the black window open.** It is the server. Closing it stops the app.

4. To stop the app: click the black window and press **Ctrl+C** (or just close the window).

Next time, you only need step 2.4: make sure PostgreSQL is running (it starts automatically with Windows) and double-click **run.bat**.

2.5 Log in

The login page shows clickable cards for the main demo accounts: click a card and the form fills itself in, then click **Login**. All demo accounts use the same password: **Demo@12345**

Email	Name	Role	What you will see
officer@mplads.demo	Anita Deshmukh	Officer	Officer dashboard, upload PDF, projects and risk, complaints, inspections
head@mplads.demo	Dr. Vikram Joshi	Head Officer	Supervisory dashboard, escalated cases, appeals
agency@mplads.demo	Suresh Patil	Implementing Agency	Municipal Works Department projects (this agency owns the demo project)
agency2@mplads.demo	Meena Shinde	Implementing Agency	PWD Pune Division projects (to prove agencies cannot see each other)
citizen@mplads.demo	Ravi Kumar	Citizen	Public projects, map, complaints
citizen2@mplads.demo	Priya Sharma	Citizen	A second citizen with a few complaints
inspector@mplads.demo	Rahul Kulkarni	Officer (field inspector)	Same as Officer, plus inspections assigned to him
admin@mplads.demo	System Administrator	Admin	User management and audit overview

Four more citizens (**citizen3** to **citizen6@mplads.demo**) exist only to own the older complaint history. They log in with the same password but the login page does not list them.

Note: A new visitor can also click **Create a citizen account** on the login page. Sign-up always creates a Citizen. Officers, agencies and admins can only be created by the Admin.

Careful: After 5 wrong passwords an account is locked for 15 minutes. The Admin can unlock it at once by resetting the password.

2.6 Mac and Linux

```
python3 setup.py      # first time only (or ./setup.sh)
python3 run.py         # every time (or ./run.sh)
```

Install PostgreSQL with your package manager (**brew install postgresql**, **apt install postgresql**) and make sure it is running. Everything else is the same.

2.7 If something goes wrong

What you see	What it means and how to fix it
"python is not recognized" / "Python was not found"	Python is not installed, or the PATH box was not ticked. Reinstall Python and tick "Add python.exe to PATH".
"Could not connect to PostgreSQL"	PostgreSQL is not running, or the password/port is wrong. Start the postgresql service (services.msc) and run <code>python setup.py --force</code> .
"permission denied to create database"	The PostgreSQL user you typed is not allowed to create databases. Use the <code>postgres</code> user, or create an empty database yourself and type its name at the last question.
The browser shows "This site can't be reached"	The server is not running. Double-click run.bat and keep its window open.
"Address already in use" / port 8000 busy	Another program uses port 8000. In a command window in the folder, type <code>set PORT=8010</code> then <code>python run.py</code> and open <code>http://localhost:8010</code> .
Login says "database error"	The server cannot reach PostgreSQL (it stopped). Start the postgresql service. No data is lost.
The map area is grey	The map tiles come from the internet. Connect to the internet; everything else works offline.
You changed something and want the original demo data back	Double-click <code>reset_data.bat</code> (type YES when asked). See Part 11.

More problems and answers are in Part 13.

Part 3 - Your first 30 minutes: a guided tour

Follow these tours in order. Each one is a few minutes. They use the demo accounts from section 2.5. Do the Officer tour first, because it creates the demo project that the other tours look at.

Tip: You can play freely. If you ever want to go back to the starting state, double-click `reset_data.bat`.

Tour 1 - The Officer: from a PDF to a decision

1. Log in as **officer@mplads.demo**. You land on the **Officer Dashboard** automatically - the system read your role from the database. You see eight number cards (Total Projects, Projects Under Review, High Risk, Critical Risk, Pending Verification, Pending Complaints, Pending Inspections, Pending Agency Requests) and three charts.
2. Click **Upload Project PDF** in the left menu. On the right there is a list of two sample PDFs. Click **Download** next to `MPLADS-PN-001_Progress_Report.pdf`.
3. Drag that downloaded file into the dashed box (or click the box and pick the file). Click **Upload & Extract**. Wait a few seconds.
4. You now see the **Extraction Preview**: a table with every field, its value, a **Confidence** badge (High / Medium / Low / Missing) and the **Source page** it came from. For example "Project ID - MPLADS-PN-001 - High - Page 1" and "Progress - 62% - Medium - Page 3". On the right is the **Validation** box: it says all checks passed for this clean PDF.
5. Click **Verify & Analyze**. A box asks for confirmation and lets you publish the project to citizens. Click **Verify & Analyze** again.
6. The **risk result page** opens. You see **Risk Score 87/100 - Critical**, status **Officer Review Required**, and a list of the reasons with points: the machine-learning "unusual pattern" signal, the 180-day delay, progress below the expected level, unusual expenditure pattern, budget revisions, extension requests, verified complaints, agency history, payment pattern. A yellow strip says these are prototype thresholds. A grey line says the score is not proof of wrongdoing.
7. Click the tabs: **Historical Comparison** (142 projects compared; delay of 180 days versus about 55 for similar projects; four charts), **Extracted Data**, **Source Documents** (open the original PDF), **Agency History**, **Citizen Feedback** (the four complaints that were waiting for this project are already linked) and **Audit Trail** (every step so far).
8. Click **Record decision**. Choose **Escalate to Head Officer**, type a reason of at least 10 characters, tick the evidence you reviewed, and give a follow-up date or action. Click **Review & submit**, then **Confirm & record**. The status becomes "Escalated to Head Officer".
9. Click the bell icon at the top. Notifications are coloured: blue = information, yellow = review, orange = action required, red = overdue or critical.

Note: Why not analyse straight away? Because a wrong number in the PDF would give a wrong score. The rule is: **no analysis on unverified data**. The officer is the human check.

Tour 2 - The messy PDF: errors, warnings and corrections

1. Still as the Officer, go to **Upload Project PDF** again and upload the second sample, `MPLADS-PC-002_Field_Report_messy.pdf`.
2. The preview now shows problems. There is **one error in red** (progress is 108%, which is impossible) and about **nine warnings in yellow** (the agency name is spelled differently, the constituency does not usually belong to

that district, a required document is not mentioned, and so on). Fields that need attention have a small warning icon. **Verify & Analyze** is disabled while an error exists.

3. Click **Edit Data**. Change **Actual Progress** to 58 and **Planned Progress** to 70, and fill in a **Sanction Date**. Click **Save changes**. The error disappears and a message says the corrections were written to the audit trail.
4. Click **Verify & Analyze**. This time the box asks you to tick "I have reviewed the warnings and want to proceed". Tick it and continue. This project gets a much lower score (about 56, Medium) - a good contrast with the first project.

Tour 3 - The Head Officer: supervision

1. Log out (top right) and log in as **head@mplads.demo**. The menu is different and the dashboard is now the **Supervisory Dashboard**: Escalated projects, High-risk cases, Critical cases, Pending supervision, Inspection issues, Appeals, Reopened cases and a 7-day audit summary.
2. Click **Escalated & High-Risk**. The project you escalated is in the list. Click it to open the same case page the officer used, but with a **Supervisory action** button instead of Record decision.
3. Click **Supervisory action**. The choices are Acknowledge, Request Evidence, Reopen, Assign Inspection, Add Supervisory Note, Escalate Further and Close Review. Try **Acknowledge** (give a reason and tick one evidence box).
4. Go back to the dashboard and use the box **Run scheduled supervisory checks**. Pick a date a month in the future in **Simulate date** and click **Run checks now**. The app raises "inspection overdue", "case unresolved" and "repeated risk pattern" notifications. Running it twice does not create duplicates.
5. Open **Complaints & Appeals**. Appeals from citizens appear here; open one and decide to uphold or reject it.

Tour 4 - The Agency: a restricted view

1. Log in as **agency@mplads.demo** (Municipal Works Department). Note what is **missing**: there are no risk scores, no other agencies' projects, no citizen names, no audit trail.
2. Open **Assigned Projects**, then **Community Hall - Kothrud**. There are four tabs: Overview, Progress reports, Documents, Requests.
3. In **Progress reports** click **Submit progress report**. Try to enter a progress lower than the current one: the server refuses with a clear message. Enter a higher value with an expenditure and a delay explanation.
4. In **Requests** ask for an **extension** (new deadline plus justification) and a **budget change**. Each asks for confirmation. Later the Officer approves or rejects them.
5. In **Documents** upload a photo. Tick "make public for citizens" if you want citizens to see it. Uploading a file with the same name creates version 2.
6. Open **Complaints** and **Clarifications** in the left menu to answer verified complaints and officer questions.

Note: Try to type an officer address such as **/officer/audit** in the browser address bar while logged in as the agency. You are sent back to your own dashboard - and even if you call the server directly it answers "403 forbidden". Access is enforced in two places.

Tour 5 - The Citizen: the public side

1. Log in as **citizen@mplads.demo** (or click **Create a citizen account** to make your own).
2. The dashboard shows public projects, a map (green = on track, orange = delayed, blue = completed) and featured projects. Open **Find Projects** and try the search and filters. Open a project to see its progress, public finances, timeline, photos and reviews.

3. Click **Report a problem / Submit complaint**. Choose a category, describe the problem, attach a photo, tick **Submit anonymously** if you like, and submit. You get a **Tracking ID**.
4. Open **My Complaints** and click your complaint: an 8-step tracker shows where it is (Submitted, Tracking ID issued, Screening, Assignment, Investigation, Resolution, Citizen feedback, Appeal or Closed).
5. To see the complaint through: log in as the Officer and **Screen** it (verify), as the Agency and **respond**, as the Officer and **Resolve**, then as the Citizen give **feedback**. If you say you are not satisfied you can **Appeal**, and the Head Officer decides.

Tour 6 - The Admin: users and roles

1. Log in as **admin@mplads.demo**. The only pages are **Users & Audit** and Notifications.
2. Click **Add user**. Choose a role. For an Implementing Agency you must choose which agency it belongs to. Leave the password empty to get a one-time generated password shown once on screen with a Copy button.
3. Use the pencil icon to **edit** a user and **change the role**. It takes effect immediately, even for someone logged in. Use the key icon to **reset a password** and the red icon to **disable** an account (the person is signed out at once).
4. The list at the bottom shows every account change from the audit trail.

Tip: You are now familiar with all screens. The rest of this guide explains how each part works inside.

Part 4 - How the project is built

4.1 The big picture

The project has three main layers that talk to each other, plus a folder for uploaded files:



The **browser part** (the frontend) is a React application. It draws the screens and asks the server for data. It never touches the database. The **server part** (the backend) is a Python program using FastAPI. It has about 64 "endpoints" - addresses such as `/api/projects` that the frontend calls. Each endpoint checks who is calling, checks their permission, reads or writes the database, and answers in JSON. The **database** is PostgreSQL and holds all data.

When you double-click run.bat one program starts: the FastAPI server. It also serves the ready-made screens, so a single address (`http://localhost:8000`) gives you both the screens and the data.

4.2 The technologies, and why each one is there

Technology	What it is	What it does here
React 18	A JavaScript library for building screens	All the pages you click (33 of them)
Vite	A build tool	Turns the React source into the ready-made files in <code>frontend/dist</code>
Tailwind CSS	A styling toolkit	Colours, spacing, responsive layout (works on phones and tablets)
Recharts	A chart library	Bar charts, donut chart, trend chart
Leaflet	A map library	The project map (uses OpenStreetMap tiles)
FastAPI	A Python web framework	The server and its 64 endpoints
SQLAlchemy	A database toolkit for Python	Defines the 18 tables in <code>models.py</code> and talks to PostgreSQL
Pydantic	A data-validation library	Checks every request body (for example that a password is long enough)
PostgreSQL	A database server	Stores everything; also refuses to change audit records
PyJWT + bcrypt	Login security	Signed login tokens and safely hashed passwords
pdfplumber + PyMuPDF	PDF readers	Read text and tables from the uploaded PDF (Tesseract OCR is an optional extra for scanned pages)
pandas + NumPy	Data analysis	Prepare the history tables for the comparison

Technology	What it is	What it does here
scikit-learn	Machine learning	The Isolation Forest that spots unusual projects
pytest	A test runner	59 automatic tests

4.3 What happens when you click a button

Take the most important click: the officer presses **Verify & Analyze**. Here is the journey, step by step, with the file that does each step:

1. **Browser** (`frontend/src/pages/officer/ExtractionPreview.jsx`) sends **POST** `/api/extractions/{id}/verify` with the login token in the request header, using `lib/api.js`.
2. **FastAPI** (`backend/app/routers/extractions.py`) receives it. `deps.py` reads the token, finds the user in the database, loads the user's role and permissions from the `roles_permissions` table, and checks the user has `project:verify`. If not: "403 forbidden".
3. **Validation** (`services/validation.py`) re-checks every field. If there are errors the server answers 422 and nothing is saved. If there are warnings and the officer has not ticked the box, it answers 409.
4. **Storage**: a `projects` row is created, payments are saved, the PDF is attached to the project, and complaints that were waiting under this project code are linked.
5. **Analysis** (`services/risk_service.py`) loads history from the database, calls the `ml/` package to compare and score, and saves a `risk_analysis` row.
6. **Notifications** (`services/notify.py`) create rows in `notifications` for the officer and the Head Officer.
7. **Audit** (`services/audit.py`) writes audit rows for the verification, status change, risk analysis and each notification, all in the same database transaction. Either everything is saved or nothing is.
8. **Answer**: the server returns the result as JSON; the browser opens the risk result page.

4.4 Login and security in plain words

- **Password storage.** Passwords are never stored. Only a bcrypt hash is stored. Even the Admin cannot read a password.
- **Login token.** After a correct login the server gives the browser a signed token (a JWT) that contains only the user's ID and an expiry (60 minutes). The browser sends it with every request. The token does **not** contain the role, so nobody can edit a token to become an officer: the server looks the role up in the database every time.
- **Logout is real.** Logging out puts the token on a "revoked" list, so it stops working even before it expires.
- **Two locks on every door.** The frontend hides pages you should not see. The backend independently refuses forbidden requests. If someone types a URL by hand or calls the API directly, the backend still says no.
- **Different answers for different roles.** A citizen asking for a project gets a smaller answer than an officer: risk scores, internal status and officer notes are simply not included (`services/serializers.py`).
- **Identity masking.** Officers see complaint authors as `Ra* Ku*`. Anonymous complaints show "Anonymous" to everyone except the author.
- **File safety.** Uploads are checked by content, not just by name; only PDF/JPG/PNG/GIF/WebP (and MP4 for complaint videos) are accepted; size limits apply; files are stored under random names; PDFs with JavaScript or password protection are rejected.
- **Lock-out.** Five wrong passwords lock the account for 15 minutes.
- **Tamper-evident audit.** Every audit row stores a fingerprint that includes the previous row's fingerprint. The database also blocks any UPDATE, DELETE or TRUNCATE on the audit table. The Head Officer's **Verify audit chain** button recomputes the chain and reports "Intact" or "Broken".

Part 5 - Folder-by-folder map

This is the whole project folder. Folders you can safely ignore when you only want to run the app are marked "(generated)".

```
mplads-risk-intelligence-platform/
|-- setup.bat / setup.sh / setup.py      first-time setup (Part 2)
|-- run.bat / run.sh / run.py           start the app
|-- reset_data.bat / reset_data.sh      wipe the database and reload the demo data
|-- package_for_sharing.py              makes a clean zip of the project
|-- requirements.txt                    list of Python packages
|-- package.json                        shortcut commands (optional)
|-- .env.example / .env                 settings template / your real settings (secret!)
|-- README.md                           short summary
|-- MPLADS_Project_Guide.docx / .pdf    this guide
|-- backend/                            the server
|   |-- app/      main.py config.py database.py models.py schemas.py security.py deps.py
|   |             permissions.py demo_accounts.py
|   |             |-- routers/          the API endpoints, one file per area
|   |             |-- services/        the working parts (PDF, risk glue, audit, notifications
... )
|   |-- tests/          59 automatic tests
|-- ml/                 comparison + Isolation Forest + risk engine
|-- database/           seed.py, generate_sample_pdfs.py, export_schema.py,
schema.sql
|-- frontend/           the screens
|   |-- src/    main.jsx App.jsx index.css lib/ context/ components/ pages/
|   |-- dist/          (generated) the ready-made screens the server serves
|   |-- package.json vite.config.js tailwind.config.js index.html
|-- demo_pdfs/          two sample PDFs (regenerated when the server starts)
|-- uploads/            uploaded PDFs, photos, evidence (created as you use the
app)
|-- .venv/              (generated) private Python packages
```

5.1 Files in the main folder

File	What it is for
setup.py (setup.bat / setup.sh)	One-time setup: creates .venv , installs packages, asks for PostgreSQL details, creates the database, writes .env , seeds the data. Options: --force (redo the questions), --yes (accept defaults), --host --port --user --password --dbname .
run.py (run.bat / run.sh)	Starts the server with uvicorn on port 8000 (change with the PORT variable) and opens the browser.
reset_data.bat / .sh	Asks for confirmation, then runs python -m database.seed --reset : deletes all data and reloads the demo data.
package_for_sharing.py	Builds a clean zip without your .env , .venv , node_modules and uploaded files.
requirements.txt	The Python packages (FastAPI, SQLAlchemy, psycopg2, PyJWT, bcrypt, PyMuPDF, pdfplumber, pandas, numpy, scikit-learn, reportlab, pytest ...).

File	What it is for
.env	Your private settings, created by setup. Contains the database password and a secret key. Never share it. See the table in section 5.2.
.env.example	The template that setup copies to make .env .
package.json	Shortcut commands for people who use Node (optional).

5.2 The settings file (.env)

Setting	Default	Meaning
DATABASE_URL	set by setup	How the server connects to PostgreSQL: <code>postgresql+psycpg2://USER:PASSWORD@HOST:PORT/DATABASE</code>
JWT_SECRET	random	Secret used to sign login tokens. If you change it everyone is logged out.
JWT_EXPIRE_MINUTES	60	How long a login lasts
CORS_ORIGINS	localhost:5173, localhost:8000	Web addresses allowed to call the API from a browser
UPLOAD_DIR	./uploads	Where uploaded files are stored
MAX_UPLOAD_MB	10	Biggest file allowed
SEED_ON_STARTUP	true	If the database has no users when the server starts, load the demo data. Also regenerates the sample PDFs each start.
DEMO_PASSWORD	Demo@12345	Password given to demo users when seeding (changing it later does not change existing users)
SHOW_DEMO_CREDENTIALS	true	Show the demo account cards on the login page. Set to false for a real deployment.
ENABLE_SCHEDULER	true	Run the supervisory checks automatically every 6 hours
ALLOW_SIGNUP	true	Let visitors create Citizen accounts
APP_ENV	development	When set to production the interactive API documentation page is switched off

Note: After you change **.env**, stop the server (Ctrl+C) and run **run.bat** again.

Part 6 - The database

All data lives in one PostgreSQL database (default name **mplads**). The tables are created automatically by the server from the definitions in **backend/app/models.py**; you never create them by hand. A ready-to-read copy of the table definitions (SQL) is in **database/schema.sql**.

6.1 The 18 tables

Table	What it stores	Important columns
users	Everyone who can log in	user_id, name, email, password_hash (never the password), role, agency_id, department, district, constituency, is_inspector, status (Active/Disabled), last_login, failed_attempts, locked_until
roles_permissions	Which role may do what (loaded from permissions.py)	role_name, permission, description
revoked_tokens	Login tokens that were logged out	jti (token id), expires_at
agencies	The 8 implementing agencies and their track record	name, district, total_projects, completed_projects, delayed_projects, average_delay, complaints, inspections, previous_high_risk_cases, historical_performance
projects	Every project (historical and live)	project_code, name, category, location, latitude, longitude, district, constituency, agency_id, sanctioned_amount, released_amount, expenditure, remaining_amount, progress, planned_progress, start_date, deadline, revised_deadline, completion_date, delay_days, budget_revisions, extension_requests, status, verification_status, is_public, is_historical, case_level
payments	Payment schedule of a project	project_id, paid_on, amount, description, reference
progress_reports	Progress snapshots over time (agency reports and historical stages)	project_id, report_date, progress, planned_progress, expenditure, delay_explanation, notes
extractions	A PDF upload while it is still being checked	document_id, status (Pending Verification / Verified / Discarded), fields (JSON: value, original, confidence, source_page, edited for every field), payments, milestones, validation (JSON), project_id once verified
documents	Files: source PDFs, photos, evidence	project_id, document_type, file_name (display), file_path (random name on disk), sha256, version, is_public, uploaded_by
reviews	Citizen ratings and comments	project_id, citizen_id, rating (1-5), comment
complaints	Citizen complaints and their whole life	tracking_id, project_id or project_code_ref, citizen_id, category, description, anonymous, status, serious, screening_status, screening_note (internal), response, resolution, feedback_satisfied, appeal, appeal_status

Table	What it stores	Important columns
agency_requests	Extension and budget-change requests	project_id, agency_id, request_type, requested_deadline, requested_amount, justification, status, decision_note
clarifications	Officer questions to agencies	project_id, agency_id, question, response, status (Open/Responded), due_date
risk_analysis	Every risk analysis ever run	project_id, risk_score, risk_level, contributing_factors (JSON), historical_comparison (JSON), anomaly_score, rule_score, model_version, input_snapshot, timestamp
inspections	Inspection requests and reports	project_id, requested_by, assigned_to, priority, status, scheduled_date, due_date, findings, recommendation, outcome
decisions	Officer decisions and Head Officer actions	project_id, user_id, role, decision, reason, evidence_reviewed, follow_up_date, action, previous_status, new_status
notifications	The bell notifications	recipient_id, type, message, priority (info/review/action/critical), status (Unread/Read), project_id, dedupe_key
audit_logs	The permanent, tamper-evident event log	user_id, role, action, entity_type, entity_id, project_id, previous_value, new_value, reason, timestamp, prev_hash, record_hash

The full column list of every table is in Appendix A at the end of this guide.

6.2 How the tables connect

- **users** may belong to one **agency** (only Implementing Agency users do).
- A **project** belongs to one **agency** and has many **payments**, **progress_reports**, **documents**, **reviews**, **complaints**, **risk_analysis** rows, **inspections**, **decisions**, **agency_requests** and **clarifications**.
- An **extraction** belongs to one **document** (the uploaded PDF). When the officer verifies it, a **project** is created and the extraction points to it.
- A **complaint** can point to a project by **project_id**. Complaints filed against a project code that does not exist yet keep the code in **project_code_ref** and are linked automatically when that project is verified. This is how the demo project already has four complaints when it arrives.
- A **decision** remembers which **risk_analysis** it was based on.
- **audit_logs** and **notifications** refer to users and projects loosely (by number) so history is never lost.

6.3 Looking inside the database

PostgreSQL comes with a program called **pgAdmin** (Start menu). Add a server: host **localhost**, port **5432**, user **postgres**, your password; then open Databases -> mplads -> Schemas -> public -> Tables and right-click a table -> **View/Edit Data -> All Rows**.

Or use the command line tool **psql** (installed with PostgreSQL):

```
"C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -d mplads

\dt                                -- list the tables
SELECT email, role, status FROM users ORDER BY role;
SELECT project_code, name, status, progress, delay_days FROM projects WHERE is_historical = false;
```

```
SELECT p.project_code, r.risk_score, r.risk_level
      FROM risk_analysis r JOIN projects p USING (project_id) ORDER BY r.risk_id DESC LIMIT 10;
SELECT audit_id, user_name, role, action, entity_id, timestamp FROM audit_logs ORDER BY
audit_id DESC LIMIT 20;
\q                                     -- quit
```

Note: The number **17** in the folder path is the PostgreSQL version. If you installed a different version, use that number (look in **C:Program FilesPostgreSQL**).

Careful: Look, but do not edit the **audit_logs** table: the database will refuse (that is the point). Editing other tables by hand can leave the data inconsistent - use the app or **reset_data.bat** instead.

Part 7 - Backend modules (the server)

The server is the folder **backend/app**. It has three kinds of files: **core files** (settings, database, security), **routers** (the API endpoints) and **services** (the working parts the routers call). Every file starts with a short comment saying what it is for.

7.1 Core files

File	What it does
main.py	Creates the FastAPI application. Adds security headers and friendly error handling (no stack traces are ever shown), connects all the routers, serves the ready-made screens, and on start-up prepares the database (init_database), regenerates the sample PDFs and starts the 6-hourly scheduler.
config.py	Reads the settings from .env (database URL, secret, limits ...) into one Settings object.
database.py	Creates the database connection pool and the session used by every request.
models.py	Defines all 18 tables as Python classes (SQLAlchemy). This is the single source of truth for the database structure.
schemas.py	Defines the shape and rules of every request body (Pydantic): for example a complaint description must be 15-4000 characters; a password needs a letter and a number.
security.py	Password hashing (bcrypt), creating and reading login tokens (JWT), the password rule, the one-time password generator.
deps.py	The "gatekeeper". get_current_user reads the token, loads the user and their permissions from the database; require("permission") blocks anyone without that permission.
permissions.py	The list of roles, the 31 permissions, which role has which, and the list of project statuses. Loaded into the roles_permissions table.
demo_accounts.py	The list of demo users created by the seed, and which ones show on the login page.

7.2 Routers - every API endpoint

A router is a file that groups related endpoints. Below, each router has a one-line summary and a table of its endpoints. "Who may call it" is the permission the server checks; **any logged-in user** means the endpoint checks the role inside (for example a citizen and an officer both call **GET /api/projects** but receive different data).

routers/auth.py

Login, logout, sign-up, "who am I", change password, public settings.

Endpoint	Who may call it	What it does
GET /api/auth/config	anyone	Tells the login page whether to show demo accounts and the sign-up link
POST /api/auth/register	anyone	Citizen self-sign-up (always creates a Citizen, logs the user in)

Endpoint	Who may call it	What it does
POST /api/auth/login	anyone	Check email/password, give back a login token (locks after 5 failures)
POST /api/auth/logout	any logged-in user (role decides what they get)	Revoke the current token
GET /api/auth/me	any logged-in user (role decides what they get)	Who am I? Returns my details, role and permissions
POST /api/auth/forgot-password	anyone	Placeholder - no email is sent in the prototype
POST /api/auth/change-password	any logged-in user (role decides what they get)	Change my own password (needs the current one)

routers/users.py

Administrator user management (create, edit, change role, disable, reset password).

Endpoint	Who may call it	What it does
GET /api/users	user:admin	Admin: list/search users
POST /api/users	user:admin	Admin: create a user (any role)
PUT /api/users/{user_id}	user:admin	Admin: change name, role, agency, status (disable/enable)
POST /api/users/{user_id}/reset-password	user:admin	Admin: set or generate a new password for a user

routers/projects.py

Project lists and details, risk results, historical comparison, reviews.

Endpoint	Who may call it	What it does
GET /api/projects	any logged-in user (role decides what they get)	List projects; the answer is filtered and shaped by role (public / own agency / all)
GET /api/projects/{project_id}	any logged-in user (role decides what they get)	One project; citizens get the public view, agencies their own view, officers everything
GET /api/projects/{project_id}/risk	project:view_risk	Latest risk analysis with all factors, plus history of earlier analyses
POST /api/projects/{project_id}/risk-analysis	project:run_analysis	Run the risk analysis now for a verified project
GET /api/projects/{project_id}/historical-comparison	project:view_risk	Comparison with historical projects (stored or computed now)

Endpoint	Who may call it	What it does
GET /api/projects/{project_id}/agency-history	project:view_risk	The agency's track record and its other projects
GET /api/projects/{project_id}/feedback	complaint:review	Reviews and complaints about the project (identities masked)
GET /api/projects/{project_id}/reviews	any logged-in user (role decides what they get)	Citizen reviews of a project
POST /api/projects/{project_id}/reviews	review:submit	A citizen posts a 1-5 star review (once per project)

routers/extractions.py

The PDF pipeline: upload, preview, corrections, verification.

Endpoint	Who may call it	What it does
POST /api/projects/upload-pdf	project:upload_pdf	Upload a project PDF: validate, save, extract fields, validate the data
GET /api/extractions	project:upload_pdf	List extractions waiting for (or past) verification
GET /api/extractions/{extraction_id}	project:upload_pdf	One extraction with every field, confidence, source page and validation
PUT /api/extractions/{extraction_id}	project:verify	Save officer corrections (each change is audited) and re-validate
POST /api/extractions/{extraction_id}/discard	project:verify	Throw an extraction away without creating a project
POST /api/extractions/{extraction_id}/verify	project:verify	"Verify & Analyze": store the project, then run the analysis

routers/documents.py

Uploading and downloading photos and documents.

Endpoint	Who may call it	What it does
POST /api/projects/{project_id}/documents	document:upload	Upload a photo/PDF for a project (versioned)
GET /api/documents/{document_id}/file	any logged-in user (role decides what they get)	Download or view a stored file (permission-checked)

routers/complaints.py

The whole complaint workflow.

Endpoint	Who may call it	What it does
POST /api/complaints	complaint:submit	Citizen files a complaint with up to 3 photos/videos
GET /api/complaints/categories	any logged-in user (role decides what they get)	The list of complaint categories
GET /api/complaints	any logged-in user (role decides what they get)	List complaints (citizen: own; agency: verified ones on its projects; officer/head: all)
GET /api/complaints/{complaint_id}	any logged-in user (role decides what they get)	One complaint, shaped for the caller's role
PUT /api/complaints/{complaint_id}	any logged-in user (role decides what they get)	Move a complaint along: screen, investigate, respond, resolve, feedback, appeal, decide appeal, close

routers/agency.py

Agency progress reports, extension/budget requests, clarifications.

Endpoint	Who may call it	What it does
POST /api/projects/{project_id}/progress-reports	project:update_progress	Agency reports progress and expenditure (updates the project)
GET /api/projects/{project_id}/progress-reports	any logged-in user (role decides what they get)	History of progress reports for a project
POST /api/projects/{project_id}/requests	request:submit	Agency asks for an extension or a budget change
GET /api/requests	any logged-in user (role decides what they get)	List extension/budget requests (agency: its own; officer: all)
PUT /api/requests/{request_id}	request:review	Officer approves or rejects a request (approval changes the project)
GET /api/clarifications	any logged-in user (role decides what they get)	List officer questions to agencies
POST /api/clarifications	clarification:request	Officer/Head asks an agency a question
PUT /api/clarifications/{clarification_id}/respond	clarification:respond	Agency answers the question

routers/inspections.py

The inspection workflow.

Endpoint	Who may call it	What it does
GET /api/inspections	inspection:view	List inspections
GET /api/inspections/{inspection_id}	inspection:view	One inspection with evidence
POST /api/inspections	inspection:request	Officer requests an inspection
PUT /api/inspections/{inspection_id}	inspection:view	Assign, schedule, complete, report, action pending, close (rules depend on role and state)
POST /api/inspections/{inspection_id}/evidence	inspection:conduct	Inspector attaches photos/PDF to an inspection

routers/decisions.py

Officer decisions and Head Officer actions.

Endpoint	Who may call it	What it does
GET /api/decisions/options	any logged-in user (role decides what they get)	Which decisions and evidence choices the current role can use
GET /api/decisions	any logged-in user (role decides what they get)	Decisions and supervisory actions (optionally for one project)
POST /api/decisions	any logged-in user (role decides what they get)	Record an officer decision or a Head Officer action

routers/notifications.py

The notification bell and list.

Endpoint	Who may call it	What it does
GET /api/notifications	notification:view	My notifications and unread count
GET /api/notifications/unread-count	notification:view	Just the unread count (for the bell badge)
PUT /api/notifications/read-all	notification:view	Mark all my notifications read
PUT /api/notifications/{notification_id}/read	notification:view	Mark one notification read

routers/audit.py

The audit trail and its integrity check.

Endpoint	Who may call it	What it does
GET /api/audit-logs	audit:view	Browse the audit trail with filters
GET /api/audit-logs/actions	audit:view	The list of action names present in the audit trail
GET /api/audit-logs/summary	audit:summary	Counts by action, role and day (last 14 days)
GET /api/audit-logs/verify-chain	audit:summary	Recompute the hash chain and report Intact or Broken

routers/system.py

Dashboards, reference lists, scheduled checks, demo PDFs.

Endpoint	Who may call it	What it does
GET /api/meta	any logged-in user (role decides what they get)	Reference lists: categories, districts, agencies, statuses, thresholds
GET /api/users/inspectors	inspection:assign	Officers flagged as field inspectors (for assignment)
GET /api/agencies	project:view_all	Agencies with their history statistics
GET /api/dashboard	any logged-in user (role decides what they get)	The numbers and charts for the caller's dashboard (different for every role)
POST /api/system/run-checks	system:run_checks	Run the supervisory checks now (optionally pretending it is a later date)
GET /api/demo/sample-pdfs	project:upload_pdf	List the two demo PDFs
GET /api/demo/sample-pdfs/{name}	project:upload_pdf	Download one demo PDF

7.3 Services - the working parts

File (services/)	What it does
audit.py	Writes audit records. Each record is chained to the previous one with a SHA-256 fingerprint, takes a database lock so records are written one at a time, and installs the database trigger that forbids changing or deleting audit rows. verify_chain re-checks the whole chain.
notify.py	Creates notifications for a list of people. Prevents duplicates using a dedupe_key . Also records each notification batch in the audit trail. Has helpers such as "all officers", "all Head Officers", "the users of this agency".
pdf_extract.py	Reads a PDF (pdfplumber text and tables; Tesseract OCR for scanned pages if installed) and finds the 26 project fields, the payment table and the milestone table, each with confidence and source page. See Part 8.
parsing.py	Turns text into values: "Rs. 18,50,000", "18.5 lakh" and "1.85 crore" all become 1850000; dates in many formats; percentages; counts such as "Nil" or "two".
validation.py	The pre-analysis checks: required fields, duplicate ID, dates, ranges, budget consistency, agency and location match, missing documents. Returns errors, warnings and notes. See Part 8.

File (services/)	What it does
file_storage.py	Safe file handling: checks the real file type from its first bytes, size and page limits, rejects PDFs with JavaScript or a password, stores files under random names, protects against path tricks.
risk_service.py	The bridge between the database and the <code>ml/</code> package: loads history, runs the comparison and the score, saves the analysis, updates the project status, sends notifications and writes the audit record.
stats.py	Agency statistics (delayed share, average delay, previous high-risk cases, performance) and "latest risk per project".
workflow.py	Officer decisions and Head Officer actions (what each choice changes, who is notified) and the scheduled supervisory checks.
serializers.py	Turns database rows into the JSON each role may see. Different functions for citizen, agency and officer views; masks identities.
reference.py	Fixed reference data: project categories, the demo districts with map centres, which constituencies belong to which district.

7.4 Roles and permissions

This matrix is exactly what the database contains. "Yes" means the role has the permission. Some actions also need **ownership**: an agency user can only act on projects of their own agency, a citizen only on their own complaints, an inspector only on inspections assigned to them.

Permission	Meaning	Citizen	Agency	Officer	Head	Admin
<code>project:view_public</code>	View public projects and public financial information	Yes	-	-	-	-
<code>project:view_assigned</code>	View projects assigned to own agency	-	Yes	-	-	-
<code>project:view_all</code>	View every project with internal information	-	-	Yes	Yes	-
<code>project:upload_pdf</code>	Upload project PDF and run extraction	-	-	Yes	-	-
<code>project:verify</code>	Correct extracted data and verify/store a project	-	-	Yes	-	-
<code>project:run_analysis</code>	Run historical comparison and risk analysis	-	-	Yes	Yes	-
<code>project:view_risk</code>	View risk score, factors and historical comparison	-	-	Yes	Yes	-
<code>project:update_progress</code>	Update progress/expenditure and submit progress reports	-	Yes	-	-	-
<code>document:upload</code>	Upload evidence photos/documents	-	Yes	Yes	-	-
<code>document:view_internal</code>	View internal/source documents	-	-	Yes	Yes	-
<code>review:submit</code>	Submit citizen review	Yes	-	-	-	-

Permission	Meaning	Citizen	Agency	Officer	Head	Admin
complaint:submit	Submit, track, give feedback on and appeal own complaints	Yes	-	-	-	-
complaint:respond	Respond to complaints on own agency projects	-	Yes	-	-	-
complaint:review	Screen, assign and resolve complaints	-	-	Yes	Yes	-
complaint:appeal_decide	Decide citizen appeals	-	-	-	Yes	-
request:submit	Request extension / budget change	-	Yes	-	-	-
request:review	Approve or reject agency requests	-	-	Yes	-	-
clarification:request	Request clarification from an agency	-	-	Yes	Yes	-
clarification:respond	Respond to officer clarification	-	Yes	-	-	-
inspection:request	Request an inspection	-	-	Yes	-	-
inspection:assign	Assign an inspection to an inspector	-	-	-	Yes	-
inspection:conduct	Schedule, complete and report on assigned inspections	-	-	Yes	Yes	-
inspection:view	View inspections	-	-	Yes	Yes	-
decision:record	Record officer decisions (clear/clarify/inspect/escalate/keep)	-	-	Yes	-	-
decision:supervise	Head Officer supervisory actions	-	-	-	Yes	-
decision:view	View decisions	-	-	Yes	Yes	-
notification:view	View own notifications	Yes	Yes	Yes	Yes	Yes
audit:view	View audit trail	-	-	Yes	Yes	Yes
audit:summary	View audit summaries and verify audit chain	-	-	-	Yes	Yes
system:run_checks	Run scheduled supervisory checks	-	-	-	Yes	-
user:admin	Administer users	-	-	-	-	Yes

7.5 Statuses used in the app

Thing	Statuses (in order of a normal life)
Project	Verified - Analysis Pending Officer Review Required Under Review Clarification Requested Inspection Requested Cleared - Routine Monitoring Escalated to Head Officer Escalated Further Supervisory Review Evidence Requested Reopened Closed
Extraction (PDF)	Pending Verification Verified Discarded
Complaint	Submitted Assigned Under Investigation Resolved Closed Appealed Rejected
Complaint screening	Pending Verified Rejected

Thing	Statuses (in order of a normal life)
Inspection	Requested Assigned Scheduled Completed Report Submitted Action Pending Closed
Agency request	Pending Approved Rejected
Clarification	Open Responded
User	Active Disabled

7.6 Notification types

Type	Sent when	Colour
Data verification required	A PDF was uploaded and needs checking	Orange
Analysis completed	A risk analysis finished	Blue / yellow
High-priority review	A High or Critical score was produced	Orange / red
Supervisory review	A High/Critical case needs supervision; a serious complaint; an appeal; head actions	Yellow / orange / red
Repeated risk pattern	An agency keeps producing High/Critical projects	Yellow
Citizen complaint	A complaint is filed, screened, resolved, appealed or is waiting too long	Yellow / orange
Agency clarification	An officer asks a question, an agency answers, or a request is made	Orange
Inspection requested	An inspection is requested, assigned or reported	Orange / yellow
Inspection overdue	An open inspection is past its due date (scheduled check)	Red
Escalation	An officer escalates a case, or the Head Officer escalates further	Red / orange
Decision completed	A decision or a request decision was recorded	Blue
High-risk project cleared	An officer cleared a High/Critical project (Head Officer is told)	Yellow
Case unresolved / Follow-up due	A case is open for over 14 days / a follow-up date passed (scheduled check)	Yellow / orange
Progress report / Agency response	An agency submitted a report or uploaded a document	Blue

Note: Colours follow the specification: **blue** = information, **yellow** = review, **orange** = action required, **red** = overdue or critical.

Part 8 - How the PDF is read and checked

8.1 The steps

1. **Safety checks** (`file_storage.py`): it must really be a PDF (checked from the file's first bytes), at most 10 MB and 60 pages, not password protected, not corrupted, and must not contain JavaScript, launch actions or embedded files. Otherwise the upload is refused with a plain message.
2. **Reading** (`pdf_extract.py`): for each page the text and the tables are read. If a page has almost no text (a scan) the server tries OCR (Tesseract) - if OCR is not installed the officer is told the PDF looks scanned.
3. **Finding fields**: every field has a list of label names (for example "Sanctioned Amount", "Sanctioned Cost", "Budget"). The reader looks for a label in a table row or in a line like **Label: value** and takes the value next to it. It also handles labels without a colon and a few sentences (for the project ID and the sanctioned amount).
4. **Converting values**: "Rs. 18,50,000", "18.5 lakh" and "1.85 crore" all become a number; dates in many formats become dates; "Nil" becomes 0.
5. **Confidence** is assigned to each value (next section) together with the **source page**.
6. **Derived values**: if the remaining amount is missing it is computed (sanctioned minus expenditure); the report date is taken from the end of the reporting period; the delay is computed if missing.
7. **Payments and milestones**: tables whose header contains "Amount" and "Date" become the payment schedule; tables with a "Milestone" column become milestones.
8. **Validation** (`validation.py`) runs on the values and produces errors, warnings and notes.
9. **Staging**: everything is stored in the `extractions` table. **No project exists yet.**
10. **Human verification**: the officer sees the preview, corrects values (each correction is audited with old and new value) and verifies. Only then is a `projects` row created and the analysis run.

8.2 What "confidence" means

Level	Meaning	Example
High	The exact expected label was found and the value was read cleanly	Project ID, Sanctioned Amount, Original Deadline
Medium	The label was generic ("Progress", "Budget", "Balance", "Delay"), or the value was taken from a sentence, was derived, or two different values were found	Progress: 62%
Low	The value could not be read cleanly, or came from OCR	A date written in an unusual way
Missing	The field was not found; the officer must type it or accept the gap	Sanction Date in the messy PDF

8.3 The 26 fields it tries to find

Group	Fields
Project	Project ID, Project Name, Category, Location, District, Constituency, Agency, Description, Latitude, Longitude
Financial	Sanctioned Amount, Released Amount, Expenditure, Remaining Amount, Budget Revisions (plus the payment table)

Group	Fields
Timeline	Sanction Date, Start Date, Original Deadline, Revised Deadline, Completion Date, Delay Days, Extension Requests
Progress	Planned Progress, Actual Progress, Reporting Period, Report Date (plus the milestone table)

8.4 The validation rules

Errors (block verification until corrected):

- A required field is missing: Project ID, Name, Category, Agency, District, Sanctioned Amount, Start Date, Original Deadline, Progress.
- The project ID has illegal characters, or **a project with that ID already exists** (duplicate).
- A date is invalid or absurd; the deadline is before the start; the completion date is before the start.
- Progress or planned progress is not a number or is outside 0-100.
- An amount is negative or zero (sanctioned); released is more than sanctioned; expenditure is more than sanctioned.
- The agency is not in the agency registry (it must match one of the 8 agencies).

Warnings (the officer must tick "I have reviewed the warnings"):

- Recommended fields missing: location, constituency, released amount, expenditure, planned progress, sanction date, report date.
- Another upload with the same project ID is waiting; the revised deadline is earlier than the original; the sanction date is after the start.
- Expenditure is more than the released amount; remaining amount does not equal sanctioned minus expenditure; the payment table does not add up to the expenditure.
- A completion date is given but progress is below 100%; the stated delay differs from (report date minus deadline).
- The agency name is a near match (for example "P.W.D. Pune Div." is matched to "PWD Pune Division").
- The district is unknown; the constituency does not normally belong to that district; the location text mentions a different district; the coordinates are outside Maharashtra.
- The PDF does not mention a **Sanction Order** or (when money was spent) a **Utilization Certificate**.

Notes (information only): map position will be approximated from the district centre; the PDF is stored as the source document (version 1).

Note: Scope: the reader is rule-based. It works very well for documents laid out like the samples and less well for very different layouts. That is a known limit of the prototype (Part 14).

Part 9 - How the risk score is calculated

All the maths is in the folder `ml/` (four small files). It has no database code, so it is easy to read and test on its own: `config.py` (weights and bands), `historical.py` (comparison), `anomaly.py` (Isolation Forest), `risk_engine.py` (the score). `backend/app/services/risk_service.py` feeds it data from the database.

9.1 The formula

$$\text{final score} = 0.75 \times \text{rule score} + 0.25 \times \text{anomaly score} \quad (\text{both } 0 - 100)$$

Every reason shown on screen is worth a number of **points**, and the points add up exactly to the final score. Rule reasons are worth at most (weight x 0.75) points each; the machine-learning signal is worth at most 25 points.

Score	Level	Colour
0 - 29	Low	Green
30 - 59	Medium	Yellow
60 - 79	High	Orange
80 - 100	Critical	Red

Careful: These bands are **prototype thresholds, not official government thresholds**, and the score is **not proof of fraud**. The screen says this next to every score.

9.2 The ten rule checks

Reason	Weight (max points)	What it looks at and how severity is worked out
Delay	22 (16.5)	Days late against the original deadline. 60% of the severity comes from the size of the delay (180 days or more = full), 40% from how much longer it is than similar projects (150 days longer = full).
Progress below expected level	15 (11.25)	Planned progress minus actual progress. 30 points behind plan = full severity.
Unusual expenditure pattern	15 (11.25)	60%: how much more of the budget is spent than similar projects had spent at the same stage (25 points more = full). 40%: how far spending is ahead of progress (25 points = full).
Budget revisions	8 (6)	Number of revisions. Two or more = full.
Extension requests	8 (6)	Number of extension requests. Two or more = full.
Payment pattern observations	6 (4.5)	Counts warning signs in the payment schedule (two = full): two or more payments just below Rs 5 lakh (between 4.5 and 5 lakh); most payments are round figures (multiples of Rs 50,000); three or more payments within 30 days; total paid is more than 15 points ahead of progress.

Reason	Weight (max points)	What it looks at and how severity is worked out
Agency historical delays	8 (6)	The agency's other projects: 35% share of projects that were delayed (60% delayed = full), 35% average delay above the overall average (60 days above = full), 30% previous High/Critical cases (three = full).
Verified complaints	8 (6)	Complaints an officer screened as verified. Four = full. Unverified complaints only add a tiny amount (up to 0.1).
Missing or inconsistent data	5 (3.75)	How many warnings and errors validation found. Five = full.
Previous adverse inspections	5 (3.75)	Earlier inspections with "Major Deficiencies" or "Irregularities Found" for this project or its agency. Two = full.

Each check produces a **severity between 0 and 1**. Points = severity x maximum points. The **rule score** is the weighted total scaled to 0-100. Wording on screen is deliberately neutral: "Unusual pattern detected", "Progress variance detected", "Financial variance detected", "Further verification recommended", "Priority review recommended".

9.3 The historical comparison

For the project being analysed, **historical.py** looks at every other verified project (about 190) and asks six questions. A project counts as **compared** when it matches at least two of them, and as a **peer** when it matches at least three:

- Same **category** (Road, Water Supply ...)
- Same **district**
- Same **agency**
- Similar **budget** (65% to 150% of this project's budget)
- Similar **duration** (70% to 130% of the planned duration)
- Same **size band** (Small under Rs 10 lakh, Medium 10-50 lakh, Large 50 lakh-2 crore, Very large over 2 crore)

From the peers it works out the average delay, the average share of the budget spent at a similar stage of progress (using the stage snapshots stored in **progress_reports**, within 8 points of progress), the typical number of revisions and extensions, and the agency's track record. The screen shows: **Compared projects, Same agency, Same category, Same district**, similar budget/duration/size/stage counts, **current delay vs similar-project average**, **current progress vs expected**, and **current expenditure vs average at a similar stage**, plus four charts and a table of the most similar projects.

9.4 The Isolation Forest (the "unusual pattern" detector)

An **Isolation Forest** is a machine-learning method for finding items that are unusual compared with a large group. Imagine cutting the data with random lines: a normal project sits among many others and needs many cuts to be separated; an unusual project is separated after very few cuts. The forest does this 300 times with different random cuts and averages the result.

- **What it learns from:** the whole historical database (all other verified projects), described by seven numbers: delay days, progress shortfall against plan, spending ahead of progress, budget revisions, extension requests, verified complaints and the size of the budget (on a log scale).

- **What it outputs:** the share of historical projects that are **less** unusual than this one, from 0 to 100. A value of 99 means "more unusual than 99% of history". It also names the main drivers (for example "expenditure ahead of progress, delay, budget revisions").
- **When it is skipped:** if there are fewer than 20 historical projects the detector is not used and the score is the rule score alone (the screen says so).
- **It is retrained on every analysis** (it takes well under a second) with a fixed random seed, so the same data always gives the same result.

9.5 A worked example: the demo project MPLADS-PN-001

Reason on screen	Points	Why
Unusual overall pattern vs history (Isolation Forest)	24.7 / 25	More unusual than about 99% of the 190 historical projects
180-day delay	15.4 / 16.5	180 days late; similar projects average about 55
Progress below expected level	9.8 / 11.25	62% actual against 88% expected
Unusual expenditure pattern	7.3 / 11.25	76.8% of the budget spent at 62% progress; similar projects had spent about 59.5%
Budget revisions	6.0 / 6	2 revisions
Multiple extension requests	6.0 / 6	2 requests
Verified complaints	6.0 / 6	Four screened complaints were waiting for the project and were linked when it was verified
Agency historical delays	5.5 / 6	The agency has a long record of delays
Payment pattern observations	4.5 / 4.5	Two payments just below Rs 5 lakh, and three payments within 27 days
Previous adverse inspections	1.9 / 3.75	One earlier adverse inspection on the agency
Data complete and consistent	0.0 / 3.75	No validation problems
Total	about 87 = Critical	rule score 83.1 (x0.75 = 62.3) + anomaly 98.9 (x0.25 = 24.7)

Note: The original brief showed 87 as "High", but its own bands make 80-100 "Critical", so the app shows Critical. To change the wording or the bands edit **LEVELS** in `ml/config.py`.

9.6 Recommendation text and status

Level	Recommendation shown	Project status set
Low	Routine monitoring appears sufficient	Officer Review Required (the officer still decides)
Medium	Further verification recommended; keep under monitoring	Officer Review Required

Level	Recommendation shown	Project status set
High	Priority review recommended; officer review of the evidence required	Officer Review Required; Head Officer is notified
Critical	Priority review recommended; supervisory (Head Officer) attention advised	Officer Review Required; Head Officer is notified in red

Part 10 - The frontend (the screens)

10.1 How the frontend is organised

The screens live in `frontend/src`. You do not need Node to run the project because the finished screens are in `frontend/dist`. You only need Node if you want to change a screen.

Place (frontend/src/)	What it contains
main.jsx	Starts the React application and wraps it with the login state and the pop-up message system.
App.jsx	The route table : which web address shows which page. Every page is wrapped in a guard: RequireAuth (you must be logged in) and RoleRoute (your role must be allowed - otherwise you are sent to your own dashboard). Pages are loaded only when first opened, which keeps the app fast.
index.css	The look: colours, buttons (btn-primary ...), cards, form fields, badges. Built on Tailwind CSS.
lib/api.js	The only place that talks to the server: adds the login token to every request, turns errors into friendly messages, and logs you out when the token expires. Also fetches protected images and files.
lib/nav.js	The menus for each role (each role sees different pages), where a notification click should lead, and the notification colours.
lib/format.js	Formatting: money in lakh/crore (Rs 18.5 lakh), dates, "3 h ago".
context/AuthContext.jsx	Keeps who is logged in. Provides login, register, logout, and the unread-notification count. Restores the session after a page refresh by asking the server "who am I?".
components/layout/	AppLayout (left menu, top bar with the bell, profile and logout with confirmation, mobile menu drawer) and NotificationPanel (the bell drop-down).
components/ui/	Reusable pieces used everywhere: Badges (risk badge, status badge, confidence badge), Layout (Card, KPI card, tabs, progress bar, empty state), Form fields, Modal and confirmation dialog and pop-up messages, DataTable, MapView, file/photo viewers, and small helper hooks (useApi , useAction).
components/officer/	CaseTabs (the seven tabs of the project case page), DecisionDialog (the decision and supervisory-action form), common pieces (audit table, risk donut, inspection stepper).
components/portal/	Citizen/agency pieces: ProjectCard, star ratings, MoneyInput (shows a lakh preview as you type), ComplaintStepper (the 8-step tracker), EvidenceGallery.
pages/	The 33 screens, grouped by role (see below).

Two extra safety ideas are built into the screens: **every important button asks for confirmation** first (decisions, escalation, approvals, logout, disabling users), and **every list has a loading state, an empty state and an error state** so a slow or failing server never leaves a blank screen.

10.2 Pages available to everybody

Address	Page file (pages/)	What it is for
/login	common/Login.jsx	Sign in with email or user ID and password. Shows the demo account cards (if enabled), a link to sign up and a "Forgot password" link. Shows a message if your session expired.
/register	common/Register.jsx	Citizen sign-up. Checks the password rule live and always creates a Citizen.
/forgot-password	common/ForgotPassword.jsx	A placeholder page: it explains that reset is done by the Admin.
/profile	common/Profile.jsx	Shows your own details and lets you change your password. You cannot change your own role.
/notifications	common/Notifications.jsx	All your notifications with filters (unread, and the four colours). Clicking one opens the related project or complaint and marks it read.
(any wrong address)	common/NotFound.jsx	"Page not found" with a button to your dashboard.

10.3 Citizen pages

Address	Page file	What it does
/citizen	citizen/Dashboard.jsx	Welcome page: search box, five number cards (public projects, completed, delayed, my complaints, open complaints), the project map, a chart by category and featured project cards.
/citizen/projects	citizen/Projects.jsx	Find projects: search by name/place, filter by category, district, agency, status (On track / Delayed / Completed) and progress; switch between cards and map.
/citizen/projects/:id	citizen/ProjectDetail.jsx	One project, public information only: progress, public finances (sanctioned, released, spent), timeline, location map, photos, reviews, a form to write a review, and the button to report a problem.
/citizen/complaints	citizen/Complaints.jsx	My complaints: search by tracking ID, filter by status.
/citizen/complaints/new	citizen/ComplaintForm.jsx	File a complaint: project, category, description, date, place, up to 3 photos or videos, anonymous option. Ends with the tracking ID.
/citizen/complaints/:id	citizen/ComplaintDetail.jsx	The 8-step tracker, the agency's response and the resolution, and the actions available now: give resolution feedback (Yes/No, rating, comment), then appeal if you are not satisfied. Explains "what happens next" in plain language.

10.4 Implementing agency pages

Address	Page file	What it does
/agency	agency/Dashboard.jsx	Six number cards (assigned projects, pending reports, delayed projects, open complaints, open clarifications, pending requests), a "needs your attention" list and a progress-versus-plan table.
/agency/projects	agency/Projects.jsx	The agency's own projects with search, status filter and a suggested next action.
/agency/projects/:id	agency/ProjectDetail.jsx	Four tabs. Overview: progress, finances, dates, milestones. Progress reports: history and a form (progress %, expenditure, period, delay explanation, notes). Documents: list and upload (type, description, "make public"). Requests: history plus forms for an extension and a budget change.
/agency/requests	agency/Requests.jsx	Track every extension/budget request with the officer's decision note.
/agency/complaints	agency/Complaints.jsx	Verified complaints on the agency's projects (no citizen identity). Read details and photos and send a response. Agencies can never close or delete a complaint.
/agency/clarifications	agency/Clarifications.jsx	Questions from officers, overdue ones highlighted, with a response box.

10.5 Officer pages

Address	Page file	What it does
/officer	officer/Dashboard.jsx	Eight number cards, a risk-level donut chart, projects by status, average risk by category and the most recent analyses.
/officer/upload	officer/UploadPdf.jsx	Upload a project PDF (drag and drop). Shows the steps Upload - Extract - Verify - Store - Compare - Analyse, and the two demo PDFs with Download buttons.
/officer/extractions	officer/Extractions.jsx	Verification Queue: uploads waiting for verification (or verified/discarded), with the number of errors and warnings.
/officer/extractions/:id	officer/ExtractionPreview.jsx	The key screen: every field with value, confidence and source page; Edit Data; the payment and milestone tables; the validation box; Verify & Analyze; Discard.
/officer/requests	officer/Requests.jsx	Approve or reject agency extension/budget requests (a note is required). Approval changes the project's deadline or sanctioned amount.

10.6 Pages shared by Officer and Head Officer

These screens exist once but are reached from both `/officer/...` and `/head/...`. The buttons shown depend on the role.

Address ending	Page file	What it does
/projects	shared/Projects.jsx	Projects and risk: search and filter (category, district, agency, status, risk level), sort by risk, delay or progress, option to include the historical archive. Each row shows risk badge, status and progress.
/projects/:id	shared/CaseView.jsx	The project case page. Header with the big risk score and status; buttons Re-run analysis and Record decision (Officer) or Supervisory action (Head). Seven tabs: Risk Factors, Historical Comparison (numbers and four charts), Extracted Data, Source Documents, Agency History, Citizen Feedback, Audit Trail. A side panel lists the decision history.
/complaints	shared/Complaints.jsx	Review complaints. Officer: screen (verify or reject with a required note), move to investigation, resolve, close. Head Officer: also decide appeals. Shows the internal screening note (never shown to citizens or agencies).
/inspections	shared/Inspections.jsx	The inspection workflow with a 7-step stepper and an overdue badge. Officer requests; Head assigns an inspector; the inspector schedules, completes, uploads evidence and submits the report; Officer/Head move it to Action Pending and Closed.
/decisions	shared/Decisions.jsx	A table of all decisions and supervisory actions with reason, evidence, follow-up and status change; filter by type.
/map	shared/MapPage.jsx	The project map coloured by risk level (grey = not analysed) with a side table.
/audit	shared/Audit.jsx	The audit trail with filters (action, role, project, dates). Head Officer also sees the summary charts and the Verify audit chain button.

10.7 Head Officer pages

Address	Page file	What it does
/head	head/Dashboard.jsx	Supervisory dashboard: eight number cards, the Run scheduled supervisory checks box (with a "simulate date" field), risk donut, risk trend over time, agencies with most High/Critical projects and the 7-day audit summary.
/head/cases	head/Cases.jsx	Escalated and high-risk cases in tabs: Escalated, Critical, High-risk, All requiring supervision.
/head/projects, /head/projects/:id, /head/complaints, /head/inspections, /head/decisions, /head/map, /head/audit	the shared pages above	Same as the officer versions, with the Head Officer's extra actions.

10.8 Admin page

Address	Page file	What it does
/admin	admin/Admin.jsx	User management: four number cards (users, active, disabled, audit-chain integrity), search and filters, the user table, Add user , edit/change role, reset password, disable/enable, and a list of recent account changes.

10.9 Changing the screens (needs Node.js 18 or newer)

```
cd frontend
npm install          # once: downloads the screen-building tools
npm run dev          # live editing at http://localhost:5173 (the API must be running on
:8000)
npm run build        # creates the finished files in frontend/dist that the server uses
```

Part 11 - The seed data

11.1 What "seed" means

A new database is empty: no users, no projects. **Seeding** means filling it with a complete, believable set of **synthetic** data so that every screen has something to show and the risk analysis has a history to compare against. The seed program is `database/seed.py` (about 540 lines). Nothing in it is real.

11.2 When the seed runs

- **During setup** (`setup.bat`, step 4/4): `python -m database.seed`.
- **When the server starts** with `SEED_ON_STARTUP=true` (the default): the server creates the tables and installs the audit protection. **It seeds only if the users table is empty**. If data already exists it only refreshes the roles/permissions table (so changes in `permissions.py` take effect) and leaves your data alone. It also regenerates the two sample PDFs each start.
- **When you reset** (`reset_data.bat`): `python -m database.seed --reset` first **drops every table** (all data is deleted), recreates them and seeds again.

Careful: Reset deletes everything, including any users or projects you created. Files in the `uploads/` folder are not removed automatically; you can delete the contents of that folder by hand.

11.3 What it creates, in order

#	Step	Result (approximate counts)
1	Roles and permissions	5 roles, 31 permissions written to <code>roles_permissions</code>
2	Agencies	8 agencies (table in 11.4)
3	Users	12: the 8 named demo accounts plus 4 extra citizens. All get the password from <code>DEMO_PASSWORD</code> (hashed).
4	Historical projects	176 old projects with 4 progress snapshots each (about 700 rows)
5	Live projects	14 current projects with 4 monthly reports each, photos and (for two of them) payment schedules
6	Complaints	45: 4 waiting for the demo project, 7 on live projects, 34 old closed ones
7	Quiet risk analyses	The real risk engine is run on the 14 live projects (no notifications sent)
8	Decisions and case states	4 decisions so the projects are in realistic states (cleared, kept under review, escalated, closed)
9	Inspections	3: one open and overdue on the bridge project, two closed old ones with findings
10	Requests and questions	1 open officer question (drainage project) and 1 pending extension request
11	Reviews	About 30 citizen reviews on the live projects
12	Notifications	9 starting notifications so every dashboard has content

#	Step	Result (approximate counts)
13	Agency statistics	The totals on each agency row are computed from the projects
14	Audit entry	A "Seed" record; the decisions above also wrote audit records

11.4 The eight agencies

Agency	District	Old projects	Typical behaviour in the generated history
Municipal Works Department	Pune	24	Often late (average delay about 78 days, many revisions and extensions). Owns the demo project.
PWD Pune Division	Pune	26	Average (about 34 days)
Zilla Parishad Works Pune	Pune	22	Good (about 20 days)
Pimpri Chinchwad Civic Works	Pimpri-Chinchwad	24	Average (about 45 days)
Water Supply & Sanitation Board Pune	Pune	22	Average to good (about 30 days)
Satara Rural Development Agency	Satara	20	Average (about 40 days)
Kolhapur Municipal Works	Kolhapur	20	Good (about 25 days)
Nashik District Works Cell	Nashik	18	Often late (about 70 days)

11.5 How the 176 historical projects are generated

- Project codes look like **MPLADS-PN-101**, **MPLADS-PC-102**, **MPLADS-ST-103** ... (PN = Pune, PC = Pimpri-Chinchwad, ST = Satara, KP = Kolhapur, NS = Nashik).
- Category** is picked with realistic weights (Road 27%, Water Supply 15%, School 12%, Drainage 12%, Community Hall 11%, Street Lighting 8%, Bridge 8%, Health Centre 7%). The Municipal Works Department is forced to have 13 road projects so the demo comparison has a good "same agency, same category" group.
- Budget** starts from a typical amount per category (for example Road about Rs 15 lakh, Bridge about Rs 60 lakh) multiplied by a random factor, between Rs 3 lakh and Rs 90 lakh. **Duration** scales with the budget (150-720 days).
- Delay** is drawn from a bell curve set by the agency's profile (see 11.4). About 15-30% of projects finish exactly on time. Revisions and extensions are more likely for later projects.
- About 80% are **completed** in the past (progress 100%, expenditure 93-100% of the sanction); about 20% are **ongoing** (some already past their deadline).
- Nine **unusual projects** are planted on purpose (delay 230-390 days, 3 extensions, 2-3 revisions, spending ahead of progress). They get an earlier High/Critical analysis so that agencies have a realistic "previous high-risk cases" count.
- Each project gets **4 progress snapshots** at 30%, 55%, 80% and 100% of its life. These are what the "average expenditure at a similar stage" comparison uses.
- Every 6th old project is marked public so that citizens see some completed projects too.
- The random generator uses a **fixed seed** (20260921), so the shape of the data is the same every time.

11.6 The 14 live projects

These are the "current" projects. They are written by hand in the **LIVE** list at the top of the live-projects section of **seed.py**, so each one demonstrates something. Scores are approximate.

Code	Project	Agency	What it demonstrates
MPLADS-PN-201	Community Hall - Kothrud	Municipal Works	71% done vs 85% planned, 30 days late; kept under review. Good agency demo project.
MPLADS-PN-202	Street Lighting - Baner	Municipal Works	On plan; cleared for routine monitoring; one complaint awaiting screening
MPLADS-PN-203	Drainage Upgrade - Hadapsar	Municipal Works	118 days late; open officer question; a pending complaint; scores High
MPLADS-PN-204	Bridge Works - Mula River Crossing	PWD Pune Division	The worst case: 300 days late, 3 revisions, 3 extensions, clustered payments, 3 complaints (one appeal pending), overdue inspection; Critical and already escalated to the Head Officer
MPLADS-PN-205	Road Development - Wagholi	PWD Pune Division	Healthy project (low risk)
MPLADS-PN-206	School Building - Shirur	Zilla Parishad	Healthy, almost finished
MPLADS-PN-207	Water Supply - Maval	Zilla Parishad	35% done vs 70% plan; a pending extension request; Medium
MPLADS-PC-208	Road Development - Pimple Saudagar	Pimpri Chinchwad	Completed; status Closed
MPLADS-PC-209	Health Centre - Nigdi	Pimpri Chinchwad	Healthy
MPLADS-PN-210	Water Supply - Kothrud	Water Supply Board	Healthy
MPLADS-PN-211	Drainage Upgrade - Katraj	Water Supply Board	140 days late; scores High
MPLADS-ST-212	School Building - Karad	Satara Rural Dev.	Healthy
MPLADS-KP-213	Community Hall - Shahupuri	Kolhapur Municipal	Healthy
MPLADS-NS-214	Road Development - Sinnar	Nashik Works Cell	210 days late, 3 extensions, 2 revisions; scores Critical

11.7 The special case: MPLADS-PN-001 is NOT seeded

The headline demo project **does not exist** after seeding. It appears only when you upload its PDF, exactly as a real new project would. The seed prepares for it: four verified complaints are stored with **project_code_ref = MPLADS-PN-001** and no project yet, and the Municipal Works Department is already in the registry. When the officer verifies the PDF, the four complaints are linked automatically. If the project were pre-loaded, uploading its PDF would be rejected as a duplicate.

11.8 Dates, and why the demo always looks "current"

The seed treats **three days ago** as "now" (the anchor date) and places every historical and live date relative to it. The two sample PDFs are regenerated at each server start using the same anchor. So the data always looks fresh, whichever day you run it. The **shape** (which project is late, by how many days) is the same every time; only the calendar dates move.

11.9 Changing the seed data

I want to ...	Change this in database/seed.py (or elsewhere)	Then
Add or rename agencies, or change how late they usually are	The AGENCIES table near the top (name, district, mean delay, spread, number of projects, extension rate, revision rate, code)	Reset
Change the 14 live projects	The LIVE list	Reset
Have more or fewer old projects	The n_hist number for each agency in AGENCIES	Reset
Change how unusual the planted cases are	The anomalies block in the historical-projects function	Reset
Add or change demo users	backend/app/demo_accounts.py (list of users; LOGIN_HINTS chooses which appear on the login page)	Reset
Change the demo password	DEMO_PASSWORD in .env	Reset (it is used only while seeding)
Change the sample PDFs	database/generate_sample_pdfs.py	Restart the server, or run it directly

Careful: There is currently **no one-command way to start with an empty system** (no demo data) because the app needs at least one Admin account and the seed is the only thing that creates it. To go live, seed once, then create your real users in the Admin screen and disable the demo accounts. This is listed in Part 14.

Part 12 - Everyday tasks (how-to recipes)

Commands below are typed in a Command Prompt opened **inside the project folder** (open the folder in File Explorer, click the address bar, type `cmd`, press Enter). On Mac/Linux replace `.venv\Scripts\python` with `.venv/bin/python`.

I want to ...	How
Add a user or change a role	Log in as the Admin -> Add user / pencil icon. See Tour 6.
Reset someone's password	Admin screen -> key icon. Leave the box empty to get a one-time password.
Disable someone who left	Admin screen -> red icon. They are signed out at once. Accounts are disabled, not deleted, so history stays.
Reset the Admin password if forgotten	Run <code>reset_data.bat</code> (everything returns to the demo state), or ask another admin to reset it.
Turn off public sign-up	In <code>.env</code> set <code>ALLOW_SIGNUP=false</code> , restart.
Hide the demo account cards on the login page	In <code>.env</code> set <code>SHOW_DEMO_CREDENTIALS=false</code> , restart.
Use a different port for the app	<code>set PORT=8010</code> then <code>python run.py</code>
Change the database connection	<code>python setup.py --force</code> (asks again) or edit <code>DATABASE_URL</code> in <code>.env</code>
Change risk weights or score bands	Edit <code>ml/config.py</code> (<code>RULE_WEIGHTS</code> must still add up to 100; <code>LEVELS</code> are the bands). Restart the server, then press Re-run analysis on a project. Old analyses keep their old score.
Change the wording of recommendations	<code>ml/risk_engine.py</code> , the <code>recommendations</code> dictionary
Add a project category	Add it to <code>CATEGORIES</code> in <code>backend/app/services/reference.py</code> (and to the weight tables in <code>seed.py</code> if you want demo projects of that kind)
Add a complaint category	<code>COMPLAINT_CATEGORIES</code> in <code>backend/app/routers/complaints.py</code>
Add an agency in a live system	There is no screen for it yet. Insert a row into the <code>agencies</code> table (pgAdmin), or add it to <code>AGENCIES</code> in <code>seed.py</code> and reset.
Back up the database	<code>"C:\Program Files\PostgreSQL\17\bin\pg_dump.exe" -U postgres -F c -f mplads_backup.dump mplads</code> and also copy the <code>uploads</code> folder
Restore a backup	<code>pg_restore.exe -U postgres -d mplads --clean mplads_backup.dump</code> (create the empty database first)
Look at the interactive API list	While the server runs (with <code>APP_ENV=development</code>) open <code>http://localhost:8000/api/docs</code> - you can try every endpoint in the browser
Regenerate the sample PDFs	<code>.venv\Scripts\python -m database.generate_sample_pdfs</code>
Export the database structure as SQL	<code>.venv\Scripts\python -m database.export_schema</code> (writes <code>database/schema.sql</code>)
Make a clean zip to give to someone	<code>python package_for_sharing.py</code> (leaves out <code>.env</code> , <code>.venv</code> , <code>node_modules</code> and <code>uploads</code>)

I want to ...	How
Move the project to another computer	Unzip, run <code>setup.bat</code> . To keep your data, restore a backup afterwards.

12.1 Running the automatic tests

The project has **59 automatic tests** in `backend/tests`. They start the real server code against a **separate test database** and check the rules that matter most. Your own data is never touched.

1. Create the empty test database once. In pgAdmin create a database called `mplads_test` (or in psql type `CREATE DATABASE mplads_test;`).
2. In the project folder run: `.venv\Scripts\python -m pytest backend/tests -q`
3. Wait about a minute. A healthy result ends with **59 passed**.

Test file	What it proves
<code>test_rbac_security.py</code>	Login gives the role from the database; forged tokens do not work; expired and logged-out tokens are refused; 5 wrong passwords lock the account; each role can reach only its own endpoints; citizens and agencies never receive risk or internal fields; agencies cannot see other agencies' projects; identities are masked; the audit table cannot be edited; error messages leak nothing.
<code>test_e2e_flow.py</code>	The complete story through the API: upload the PDF, see the fields and confidence, correct and validate, verify and analyse (score about 87, Critical, 142 compared), duplicate rejection, notifications, decisions, escalation, Head Officer actions, the inspection lifecycle, scheduled checks, agency workflow, citizen complaint and appeal, the messy PDF, unsafe PDFs, and the audit chain.
<code>test_accounts.py</code>	Sign-up creates only Citizens; password rules; duplicate emails; sign-up can be switched off and is rate limited; only the Admin can manage users; role changes and disabling take effect immediately; an admin cannot lock themselves out; password reset; passwords never appear in the audit log.

Part 13 - Troubleshooting and questions

13.1 Problems

Problem	Cause and fix
setup.bat closes immediately	Open a Command Prompt in the folder and run <code>python setup.py</code> to see the message. Usually Python is not installed or not on PATH.
pip cannot download packages / timeouts	You need internet during setup. Behind a company proxy set the <code>HTTPS_PROXY</code> variable, or try another network.
Setup fails at "installing dependencies" with a build error	Very new Python versions sometimes lack ready-made packages. Install Python 3.12 and run setup again (delete the <code>.venv</code> folder first).
"Could not connect to PostgreSQL"	The service is not running, the password is wrong, or the port is not 5432. Start the service (services.msc) and run <code>python setup.py --force</code> .
Your password contains @ : / or #	That is fine. Setup encodes it. If you edit <code>.env</code> by hand, encode special characters (for example @ becomes %40).
run.bat says "Not set up yet"	Run setup.bat first (it creates <code>.venv</code> and <code>.env</code>).
Login page loads but login says "database error"	PostgreSQL stopped. Start it. The server reconnects by itself.
Everything looks empty after login	The database is empty. Run <code>python -m database.seed</code> or <code>reset_data.bat</code> .
"Invalid email/user ID or password"	Check the email spelling and that you use <code>Demo@12345</code> . After 5 failures wait 15 minutes or ask the Admin to reset the password.
"Your session has expired"	Logins last 60 minutes. Log in again.
Upload says "not a valid PDF" or "corrupted"	The file is not a normal PDF. Use the two sample PDFs from the upload page to test.
Upload says "looks like a scanned document"	Text could not be read and OCR is not installed. Install Tesseract (github.com/tesseract-ocr/tesseract) and make sure it is on PATH, or use a text PDF.
Upload says "duplicate"	A project with that ID already exists. That is intended. Use <code>reset_data.bat</code> to start fresh, or use the other sample PDF.
Map is grey	No internet (map tiles). Markers still work.
The page looks broken after an update	Press Ctrl+F5 to reload without the browser cache.
Another program uses port 8000	Use <code>set PORT=8010</code> then <code>python run.py</code> .

13.2 Questions people ask

Question	Answer
Is any real data used?	No. Everything is generated. The only outside connection is the map tiles from OpenStreetMap.

Question	Answer
Where is my data?	In your own PostgreSQL database, and uploaded files in the uploads folder. Nothing is sent anywhere.
Can two people use it at the same time?	Yes, from different browsers. By default the server listens only on this computer. To let other computers on your network use it you would have to change the start command; that is not needed for a demo.
Why can an agency not see the risk score?	By design: showing an agency its own risk score could let it adjust reports to game the system. Agencies see only operational information.
Why did a score change after I pressed Re-run analysis?	The inputs changed: for example a complaint was verified, an extension was approved, or new payments were added. Every analysis is stored, so history is kept.
Why are all dates in the current year?	The seed places dates relative to "three days ago" so the demo always looks current (section 11.8).
How does the app know a complaint is "verified"?	An officer (or the Head Officer) screens it. Only verified complaints count fully in the risk score.
Who is a field inspector?	An Officer with the "Field inspector" box ticked. The Head Officer can assign inspections only to those users.
Why does deleting never appear?	Nothing is deleted from screens: users are disabled, complaints are closed, extractions are discarded. That keeps the audit trail meaningful.
Is it safe to put this on the internet?	Not as it is. It is a prototype with demo data and a shared demo password. See Part 14 for what to do first.

Part 14 - Limits and ideas for the future

14.1 What this prototype is - and is not

- It is a **complete, connected workflow** with real login, real permissions, a real database, real PDF reading and a real explainable score.
- It is **not production software**. It uses synthetic data, a shared demo password and has had no security audit or load testing.
- The score weights and bands are **prototype values**, not official. A score is a signal for human review, never a finding.

14.2 Known limits

- PDF reading is **rule-based**. It handles documents laid out like the samples very well, but a very different layout may give many "Missing" fields.
- **OCR for scanned PDFs** needs Tesseract installed and was not tested in this project.
- There is **no email sending**, so password reset is done by the Admin and notifications are in-app only.
- There is **no screen to add agencies**, and no way to start with an empty system (see section 11.9).
- Sign-up has **no email verification**.
- The map needs internet for its tiles.
- Field inspectors upload photos but there is no GPS tagging.
- The historical database is generated. With real data the weights and the comparison rules would need review with domain experts.
- Login tokens are kept in the browser tab (session storage) and last 60 minutes; there is no refresh-token mechanism.

14.3 Sensible next steps

1. Replace synthetic history with real (anonymised) records and review the weights with the people who own the process.
2. Add email (verification, password reset, notifications), and a first-admin set-up screen.
3. Add an agency-management screen and bulk import of projects from spreadsheets.
4. Improve the PDF reader with templates per document type or a document-understanding model, and test OCR.
5. Security hardening for hosting: HTTPS, a real secret store, rate limiting on login by IP, regular backups, monitoring.
6. Deploy on a server (the app is one Python process plus PostgreSQL, so it is simple to host).

Glossary

Word	Meaning
MPLADS	Member of Parliament Local Area Development Scheme (India): money given to MPs for local development works.
Implementing agency	The government body that actually builds the work (for example a municipal corporation).
Sanctioned amount	The budget approved for a project.
Released amount	Money already handed over to the agency.
Expenditure	Money the agency reports it has spent.
Lakh / crore	Indian units: 1 lakh = 100,000; 1 crore = 10,000,000.
Progress / planned progress	How much of the physical work is done / how much should be done by now.
Extraction	The automatic reading of a PDF into fields.
Confidence	How sure the reader is about a value: High, Medium, Low or Missing.
Verification	The human check of extracted values before anything is analysed.
Validation	Automatic checks (missing values, impossible values, duplicates ...).
Risk score	A number 0-100 that helps decide which projects to look at first. Not proof of wrongdoing.
Isolation Forest	A machine-learning method that finds items that are unusual compared with a large group.
Historical comparison	Comparing a project with earlier similar projects.
Escalation	Passing a case to a more senior person (the Head Officer).
Inspection	A site visit by a field inspector, with findings and an outcome.
Appeal	A citizen asking the Head Officer to review a complaint decision.
Audit trail / audit log	The permanent record of who did what and when.
Hash chain	Each audit record contains a fingerprint of the one before, so any change to old records is detectable.
Seed	Filling an empty database with demo data.
API / endpoint	The set of addresses the screens use to ask the server for data. One address is an endpoint.
REST	The style of API used here: addresses like <code>/api/projects</code> and the actions GET (read), POST (create), PUT (change).
JSON	The text format the server and the screens use to exchange data.
JWT (token)	A signed ticket the server gives you at login; the browser shows it with every request.
RBAC	Role-based access control: what you may do depends on your role.
Permission	One specific right, for example <code>project:verify</code> .
bcrypt	A method for storing passwords safely as a hash.

Word	Meaning
Router / service	Backend files: a router defines endpoints; a service does the working part.
ORM / SQLAlchemy	A tool that lets Python code use database tables as classes.
Frontend / backend	The screens in your browser / the server behind them.
React / Vite / Tailwind	The tools used to build the screens.
Virtual environment (.venv)	A private folder of Python packages for this project only.
.env file	The settings file with secrets. Never share it.
pgAdmin / psql	Programs (installed with PostgreSQL) for looking at the database.

Appendix A - Every table and column

Generated directly from `models.py`. Types are the SQLAlchemy type names (integer, string, numeric, date, datetime, jsonb ...).

Table	Columns
agencies	agency_id (integer), name (varchar), district (varchar), constituency (varchar), total_projects (integer), completed_projects (integer), delayed_projects (integer), average_delay (float), complaints (integer), inspections (integer), previous_high_risk_cases (integer), historical_performance (float)
audit_logs	audit_id (bigint), user_id (integer), user_name (varchar), role (varchar), action (varchar), entity_type (varchar), entity_id (varchar), project_id (integer), previous_value (jsonb), new_value (jsonb), reason (text), timestamp (datetime), ip_address (varchar), prev_hash (varchar), record_hash (varchar)
revoked_tokens	jti (varchar), expires_at (datetime)
roles_permissions	role_id (integer), role_name (varchar), permission (varchar), description (varchar)
users	user_id (integer), name (varchar), email (varchar), password_hash (varchar), role (varchar), agency_id (integer), department (varchar), district (varchar), constituency (varchar), is_inspector (boolean), status (varchar), last_login (datetime), failed_attempts (integer), locked_until (datetime), created_at (datetime)
projects	project_id (integer), project_code (varchar), name (varchar), category (varchar), location (varchar), latitude (float), longitude (float), district (varchar), constituency (varchar), agency_id (integer), description (text), sanction_date (date), sanctioned_amount (numeric), released_amount (numeric), expenditure (numeric), remaining_amount (numeric), progress (float), planned_progress (float), start_date (date), deadline (date), revised_deadline (date), completion_date (date), as_of_date (date), reporting_period (varchar), delay_days (integer), budget_revisions (integer), extension_requests (integer), milestones (jsonb), status (varchar), verification_status (varchar), is_public (boolean), is_historical (boolean), case_level (integer), created_by (integer), verified_by (integer), verified_at (datetime), created_at (datetime), updated_at (datetime)
agency_requests	request_id (integer), project_id (integer), agency_id (integer), request_type (varchar), requested_deadline (date), requested_amount (numeric), justification (text), status (varchar), submitted_by (integer), decided_by (integer), decision_note (text), created_at (datetime), decided_at (datetime)
clarifications	clarification_id (integer), project_id (integer), agency_id (integer), requested_by (integer), question (text), response (text), responded_by (integer), status (varchar), due_date (date), created_at (datetime), responded_at (datetime)
complaints	complaint_id (integer), tracking_id (varchar), project_id (integer), project_code_ref (varchar), citizen_id (integer), category (varchar), description (text), incident_date (date), location_text (varchar), evidence (jsonb), anonymous (boolean), status (varchar), serious (boolean), screening_status (varchar), screened_by (integer), screened_at (datetime), screening_note (text), response (text), responded_by (integer), responded_at (datetime), resolution (text), resolved_by (integer), resolved_at (datetime), feedback_satisfied (boolean), feedback_rating (integer), feedback_comment (text), appeal (text), appeal_status (varchar), appeal_decision_note (text), created_at (datetime), updated_at (datetime)

Table	Columns
inspections	inspection_id (integer), project_id (integer), requested_by (integer), assigned_to (integer), assigned_by (integer), reason (text), priority (varchar), status (varchar), scheduled_date (date), due_date (date), findings (text), recommendation (text), outcome (varchar), evidence (jsonb), requested_at (datetime), completed_at (datetime), closed_at (datetime), updated_at (datetime)
notifications	notification_id (integer), recipient_id (integer), type (varchar), message (text), priority (varchar), status (varchar), project_id (integer), entity_type (varchar), entity_id (varchar), dedupe_key (varchar), created_at (datetime), read_at (datetime)
payments	payment_id (integer), project_id (integer), paid_on (date), amount (numeric), description (varchar), reference (varchar)
progress_reports	report_id (integer), project_id (integer), agency_id (integer), submitted_by (integer), period_label (varchar), report_date (date), progress (float), planned_progress (float), expenditure (numeric), delay_explanation (text), notes (text), created_at (datetime)
reviews	review_id (integer), project_id (integer), citizen_id (integer), rating (integer), comment (text), created_at (datetime)
risk_analysis	risk_id (integer), project_id (integer), risk_score (float), risk_level (varchar), contributing_factors (jsonb), historical_comparison (jsonb), anomaly_score (float), rule_score (float), model_version (varchar), input_snapshot (jsonb), analyzed_by (integer), timestamp (datetime)
decisions	decision_id (integer), project_id (integer), user_id (integer), role (varchar), decision (varchar), reason (text), evidence_reviewed (jsonb), follow_up_date (date), action (text), risk_id (integer), previous_status (varchar), new_status (varchar), created_at (datetime)
documents	document_id (integer), project_id (integer), complaint_id (integer), inspection_id (integer), document_type (varchar), file_name (varchar), file_path (varchar), content_type (varchar), size_bytes (integer), sha256 (varchar), version (integer), is_public (boolean), description (varchar), uploaded_by (integer), uploaded_at (datetime)
extractions	extraction_id (integer), document_id (integer), uploaded_by (integer), status (varchar), fields (jsonb), payments (jsonb), milestones (jsonb), validation (jsonb), notes (jsonb), project_id (integer), created_at (datetime), updated_at (datetime), verified_by (integer), verified_at (datetime)

Appendix B - Command cheat sheet

Task	Windows	Mac / Linux
First-time setup	double-click <code>setup.bat</code>	<code>python3 setup.py</code>
Start the app	double-click <code>run.bat</code>	<code>python3 run.py</code>
Stop the app	Ctrl+C in the black window	Ctrl+C
Redo the database questions	<code>python setup.py --force</code>	<code>python3 setup.py --force</code>
Reset all data to the demo state	double-click <code>reset_data.bat</code>	<code>./reset_data.sh</code>
Seed an empty database	<code>.venv\Scripts\python -m database.seed</code>	<code>.venv/bin/python -m database.seed</code>
Run the tests	<code>.venv\Scripts\python -m pytest backend/tests -q</code>	<code>.venv/bin/python -m pytest backend/tests -q</code>
Make a clean zip	<code>python package_for_sharing.py</code>	<code>python3 package_for_sharing.py</code>
Change the port	set <code>PORT=8010</code> then <code>python run.py</code>	<code>PORT=8010 python3 run.py</code>
Rebuild the screens (Node)	<code>npm --prefix frontend run build</code>	same

Default addresses: the app <http://localhost:8000> - interactive API list <http://localhost:8000/api/docs> (development mode) - PostgreSQL localhost:5432.